

Event-Driven Serverless Order Management System

Final Project – Cloud Computing

Built on Amazon Web Services (AWS)

Team Members:

Elias Mouawad · Eran Sadgan · Daniel Gorodnitskiy

Date: May 2026

1. System Overview

This project implements a cloud-based Order Management System for an organization. Orders are managed through REST APIs, and the system reacts automatically to important events. When an order is deleted, the system asynchronously notifies subscribed users by email and backs up the deleted order to object storage, without delaying the user's request.

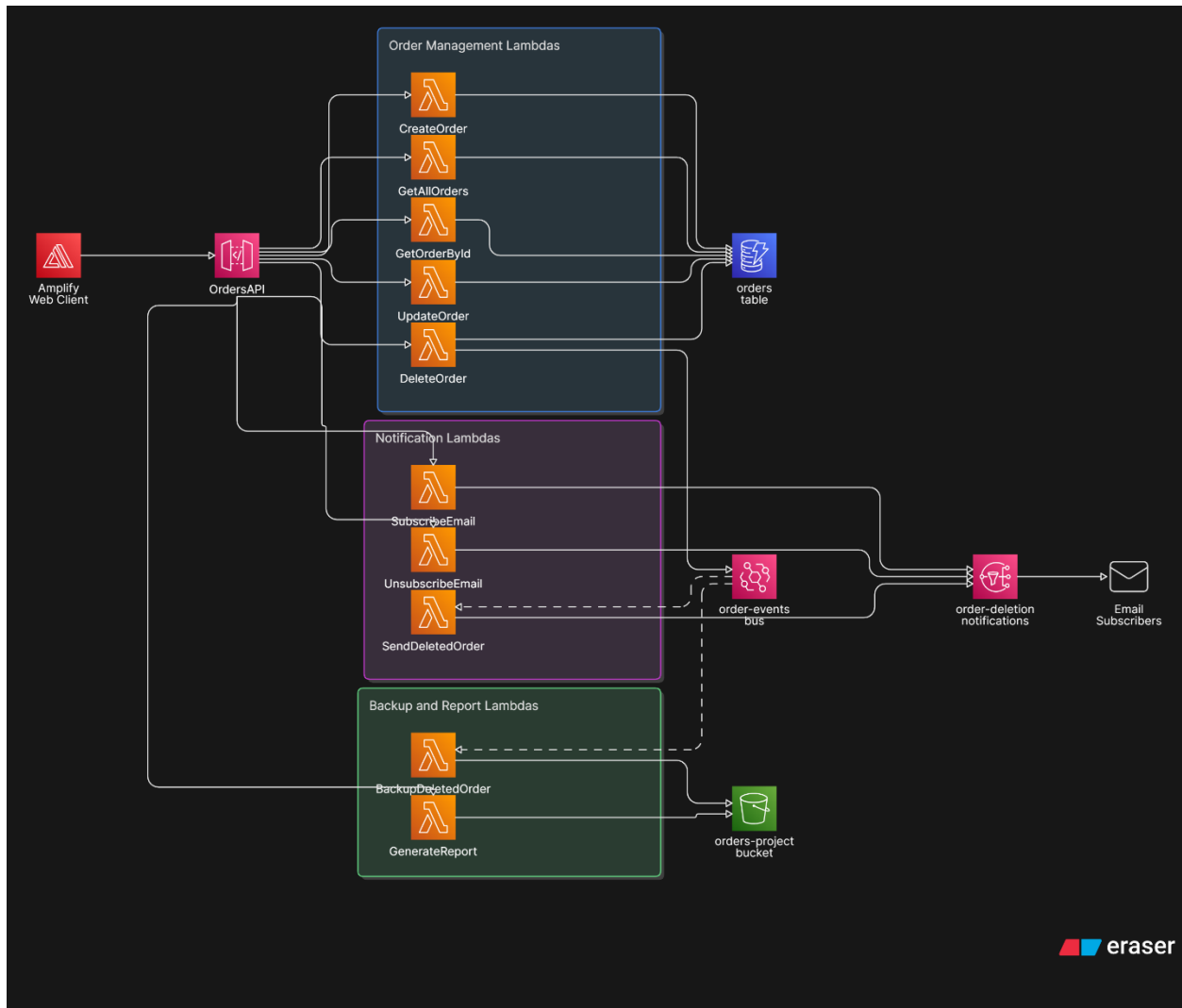
The entire solution is serverless, event-driven, and scalable. All compute runs on AWS Lambda, orders are stored in Amazon DynamoDB, deletion events are routed through Amazon EventBridge, notifications are delivered via Amazon SNS, and backups and PDF reports are stored in Amazon S3. A web client hosted on AWS Amplify consumes all the APIs.

Feature Summary

- Order management: create, get all (sorted by creation date), get one, update, delete.
- Notification subscription: register and unsubscribe an email for deletion notifications.
- Asynchronous deletion: the API responds immediately; notification and backup run in the background.
- Backup of deleted orders as TXT files in S3.
- On-demand PDF summary of all deleted orders, returned as a downloadable URL.
- Freestyle enhancement: Amazon SQS Dead Letter Queue for resilience.
- Web client on AWS Amplify that calls every API and displays the results.

2. AWS Diagram

The diagram below shows all AWS services used and how they are integrated. The web client calls API Gateway, which invokes Lambda functions. Create, read, and update operations interact directly with DynamoDB. The delete operation publishes an OrderDeleted event to a custom EventBridge bus and returns immediately. EventBridge then fans the event out to two Lambda functions in parallel: one sends an email notification through SNS, and the other writes a backup file to S3. A separate Lambda generates a PDF report from the S3 backups on demand. Failed asynchronous events are captured by an SQS Dead Letter Queue.



How the Services Integrate

- Web Client (AWS Amplify) sends HTTPS requests to API Gateway.
- API Gateway invokes the matching Lambda function for each route.
- Create / Get / Update Lambdas read and write the DynamoDB orders table.
- Delete Lambda deletes from DynamoDB, publishes an OrderDeleted event to EventBridge, and returns a response immediately.

- EventBridge rule routes the event to the notification Lambda (SNS email) and the backup Lambda (S3 TXT file).
- The PDF report Lambda reads all TXT backups from S3, builds a PDF, stores it in S3, and returns a presigned download URL.
- An SQS Dead Letter Queue receives any event that fails to be processed, preventing data loss.

3. AWS Setup per Service

Each service used in the solution is listed below with the reason it was chosen and a runnable CLI command to verify it. All commands run in the AWS Learner Lab (region us-east-1).

AWS Service	Why this service / relation to functionality	CLI command to view details
DynamoDB	Persistent NoSQL store for the orders table. Fast key-based lookups, fully managed, pay-per-request scaling.	<code>aws dynamodb describe-table --table-name orders</code>
Lambda	Serverless compute that runs all API handlers and event-driven functions; no servers to manage.	<code>aws lambda list-functions --query "Functions[*].FunctionName"</code>
API Gateway	Exposes the REST APIs and maps each route to its Lambda using proxy integration.	<code>aws apigateway get-rest-apis</code>
EventBridge	Event bus that routes the OrderDeleted event to multiple targets, enabling asynchronous processing.	<code>aws events list-rules --event-bus-name order-events-bus</code>
SNS	Sends email notifications to all subscribed users automatically when an order is deleted.	<code>aws sns list-topics</code>
S3	Object storage for deleted-order TXT backups and the generated PDF report.	<code>aws s3 ls</code>
SQS (DLQ)	Dead Letter Queue that captures failed asynchronous events (freestyle enhancement).	<code>aws sqs list-queues</code>
Amplify	Hosts the static web client over HTTPS so it can be tested via the browser.	<code>aws amplify list-apps</code>

DynamoDB Key Design (Justification)

The orders table uses `orderId` (String, a UUID) as the partition key, which uniquely identifies each order. `creationDate` (String, ISO-8601) is the sort key, which keeps the composite key unique and supports range queries by date. A Global Secondary Index (`CreationDateIndex`) allows retrieving all orders sorted by creation date efficiently, without scanning the whole table. A UUID partition key was chosen because it guarantees global uniqueness and avoids hot-partition problems.

4. APIs List

Base URL: <https://pn0ismlt6h.execute-api.us-east-1.amazonaws.com/dev>

API Name	Method	API URL	Sample Input	Sample Output
Create Order	POST	/orders	{ "price": 99.99, "orderDescription": "Test" }	{ "orderId": "...", "message": "..."} }
Get All Orders	GET	/orders	(no body)	[{ order }, { order }, ...]
Get One Order	GET	/orders/{orderId}	/orders/abc-123	{ order }
Update Order	PUT	/orders/{orderId}	{ "price": 50, "orderDescription": "New" }	{ "message": "..." }
Delete Order	DELETE	/orders/{orderId}	/orders/abc-123	{ "message": "Order deleted..." }
Subscribe	POST	/subscribe	{ "email": "user@mail.com" }	{ "message": "..." }
Unsubscribe	POST	/unsubscribe	{ "email": "user@mail.com" }	{ "message": "..." }
PDF Report	GET	/deleted-orders/report	(no body)	{ "url": "https://...pdf" }

5. Client URL (Web Application)

The client is a web application built with HTML, CSS, and JavaScript, hosted on AWS Amplify. It only calls the REST APIs and displays the returned results; all business logic is implemented in AWS services.

Web Application URL:

<https://main.d30ph7c4inun4a.amplifyapp.com>

Web Client Screenshot

The screenshot displays the 'Order Management System' web application. The interface is set against a solid blue background. At the top, the title 'Order Management System' is centered in white. Below this, there are three distinct white rectangular panels. The first panel, titled 'Freestyle Enhancement', contains a paragraph explaining the system's use of Amazon SQS Dead Letter Queues (DLQ) for handling failed EventBridge notifications. The second panel, titled 'Create / Update Order', features three input fields for 'Order ID (Leave blank to cre', 'Price', and 'Description', followed by a blue 'Save Order' button. The third panel, titled 'Notifications & Reports', includes an 'Email Address' input field, 'Subscribe' and 'Unsubscribe' buttons, and a 'Download Deleted Orders PDF' button at the bottom.

Order Management System

Freestyle Enhancement

This system uses **Amazon SQS Dead Letter Queues (DLQ)**. If the EventBridge async notification or backup targets fail to execute, the event is automatically routed to the SQS queue to prevent data loss.

Create / Update Order

Save Order

Notifications & Reports

Subscribe

Unsubscribe

Download Deleted Orders PDF

6. List of Tested Flows

The following flows were tested end to end. Each includes a short explanation and a screenshot of the steps and results.

6.1 Create Order

Entered a price and description and clicked Save Order. The new order was created in DynamoDB and appeared in the orders list, and the API response was displayed.

Code **Test** **Monitor** **Configuration** **Aliases** **Versions**

Executing function: succeeded (logs [↗](#))

▼ Details

```
{
  "statusCode": 201,
  "headers": {
    "Access-Control-Allow-Origin": ""
  },
  "body": "{ \"message\": \"Order created successfully\", \"order\": { \"orderId\": \"\\\"44c7857b-b28e-4fa5-a72b-cfcca188a195\\\"\", \"entityType\": \"\\\"ORDER\\\"\", \"creationDate\": \"\\\"2026-05-30T19:19:27.460887Z\\\"\", \"price\": \"\\\"99.99\\\"\", \"description\": \"\\\"No description provided\\\"\", \"lastModifiedDate\": \"\\\"2026-05-30T19:19:27.460887Z\\\"\" } }\""
```

Summary

Code SHA-256 srV4+FK50Db79ksFE23vZjXR+YEP4WnnutdwBWFbykQ=	Execution time 8 minutes ago
Function version \$LATEST	Request ID 759b64d3-b7c8-435d-82e6-cd9113dff175
Duration 295.73 ms	Billed duration 810 ms
Resources configured 128 MB	Max memory used 93 MB
Init duration 514.16 ms	

Log output

The area below shows the last 4 KB of the execution log. [Click here ↗](#) to view the corresponding CloudWatch log group.

```
START RequestId: 759b64d3-b7c8-435d-82e6-cd9113dff175 Version: $LATEST
END RequestId: 759b64d3-b7c8-435d-82e6-cd9113dff175
REPORT RequestId: 759b64d3-b7c8-435d-82e6-cd9113dff175 Duration: 295.73 ms Billed Duration: 810 ms Memory Size: 128 MB Max Memory Used: 93 MB Init Duration: 514.16 ms
```

6.2 Get All Orders / Get One

Clicked Refresh Orders to retrieve all orders sorted by creation date. Each order shows its ID, creation date, price, and description.

Code **Test** **Monitor** **Configuration** **Aliases** **Versions**

Executing function: succeeded (logs [↗](#))

▼ Details

```
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": ""
  },
  "body": "{ \"creationDate\": \"\\\"2026-05-28T12:03:10.077402Z\\\"\", \"lastModifiedDate\": \"\\\"2026-05-28T12:45:40.113098Z\\\"\", \"price\": \"\\\"None\\\"\", \"orderId\": \"\\\"5ad322c9-26aa-4d7e-acf9-8142792e444a\\\"\", \"orderDescription\": null, \"entityType\": \"\\\"ORDER\\\"\", \"description\": \"\\\"No description provided\\\"\", { \"price\": \"\\\"None\\\"\", \"lastModifiedDate\": \"\\\"2026-05-28T13:08:13.898778Z\\\"\", \"orderDescription\": null, \"orderId\": \"\\\"f15efe2-03d1-406a-83df-30ad000a665\\\"\", \"entityType\": \"\\\"ORDER\\\"\", \"description\": \"\\\"No description provided\\\"\", \"creationDate\": \"\\\"2026-05-28T12:08:07.708180Z\\\"\", { \"price\": \"\\\"None\\\"\", \"creationDate\": \"\\\"2026-05-28T12:43:30.222922Z\\\"\", \"orderDescription\": null, \"orderId\": \"\\\"ad77ceac-f40b-41ec-b81a-8cf6b1acab1\\\"\", \"entityType\": \"\\\"ORDER\\\"\", \"description\": \"\\\"No description provided\\\"\", \"lastModifiedDate\": \"\\\"2026-05-28T12:45:24.314072Z\\\"\", { \"creationDate\": \"\\\"2026-05-30T06:27:43.618485Z\\\"\", \"price\": \"\\\"50\\\"\", \"entityType\": \"\\\"ORDER\\\"\", \"description\": \"\\\"No description provided\\\"\", \"lastModifiedDate\": \"\\\"2026-05-30T06:27:43.618485Z\\\"\", \"orderId\": \"\\\"8bc100ed-\""
```

Summary

Code SHA-256 qt6/w7wICKCpd/e450tkyQDLABxW1A4HTu/d41P19WY=	Execution time 1 second ago
Function version \$LATEST	Request ID a8cd8df1-20c9-49cc-8fe9-11fe4d1eccea
Duration 300.90 ms	Billed duration 803 ms
Resources configured 128 MB	Max memory used 93 MB
Init duration 501.89 ms	

Log output

The area below shows the last 4 KB of the execution log. [Click here ↗](#) to view the corresponding CloudWatch log group.

```
START RequestId: a8cd8df1-20c9-49cc-8fe9-11fe4d1eccea Version: $LATEST
END RequestId: a8cd8df1-20c9-49cc-8fe9-11fe4d1eccea
REPORT RequestId: a8cd8df1-20c9-49cc-8fe9-11fe4d1eccea Duration: 300.90 ms Billed Duration: 803 ms Memory Size: 128 MB Max Memory Used: 93 MB Init Duration: 501.89 ms
```

6.3 Update Order

Entered an existing order ID with a new price/description and clicked Save Order. The order was updated and the response displayed.

The screenshot shows the AWS Lambda console interface for a function named 'Update Order'. The 'Test' tab is selected, and the function execution is successful. The response body is a JSON object with a status code of 200, headers, and a message indicating the order was updated. The summary section provides details about the function's code, version, duration, and resources. The log output shows the execution log, including the request ID, duration, and memory usage.

Code: `Test` | Monitor | Configuration | Aliases | Versions

Executing function: succeeded (logs [i](#))

Details

```
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*"
  },
  "body": "{\"message\": \"\\\"updated\\\"\"}"
}
```

Summary

Code SHA-256 mqEY73akxkjDQum7P9V8cC2eQ+1wK5SM6W67JoTIZpo=	Execution time 1 second ago
Function version \$LATEST	Request ID 5aa5cb2e-aff7-402b-9e5a-062d76df64cf
Duration 311.49 ms	Billed duration 844 ms
Resources configured 128 MB	Max memory used 93 MB
Init duration 531.98 ms	

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

```
START RequestId: 5aa5cb2e-aff7-402b-9e5a-062d76df64cf Version: $LATEST
END RequestId: 5aa5cb2e-aff7-402b-9e5a-062d76df64cf
REPORT RequestId: 5aa5cb2e-aff7-402b-9e5a-062d76df64cf Duration: 311.49 ms Billed Duration: 844 ms Memory Size: 128 MB Max Memory Used: 93 MB Init Duration: 531.98 ms
```

6.4 Delete Order (Asynchronous)

Clicked Delete on an order. The API responded immediately with a success message, while notification and backup ran asynchronously in the background.

The screenshot shows the AWS Lambda console interface for a function named 'Delete Order'. The 'Test' tab is selected, and the function execution is successful. The response body is a JSON object with a status code of 200, headers, and a message indicating the order was deleted and an event was published successfully. The summary section provides details about the function's code, version, duration, and resources. The log output shows the execution log, including the request ID, duration, and memory usage.

Code: `Test` | Monitor | Configuration | Aliases | Versions

Executing function: succeeded (logs [i](#))

Details

```
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*"
  },
  "body": "{\"message\": \"\\\"Order deleted and event published successfully\\\"\"}"
}
```

Summary

Code SHA-256 /UplUFGJ+/uNyEte1W2LJ+5CBn08PTG/Tg1H9hBAv4=	Execution time 1 second ago
Function version \$LATEST	Request ID 0e3fc64c-5b71-47bd-9414-c13cf6231b13
Duration 721.02 ms	Billed duration 722 ms
Resources configured 128 MB	Max memory used 90 MB

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

```
START RequestId: 0e3fc64c-5b71-47bd-9414-c13cf6231b13 Version: $LATEST
END RequestId: 0e3fc64c-5b71-47bd-9414-c13cf6231b13
REPORT RequestId: 0e3fc64c-5b71-47bd-9414-c13cf6231b13 Duration: 721.02 ms Billed Duration: 722 ms Memory Size: 128 MB Max Memory Used: 90 MB
```

Test event [info](#)

[Delete](#) [CloudWatch Logs Live Tail](#) [Save](#) [Test](#)

6.5 Subscribe and Email Notification

Subscribed an email address and confirmed it. After deleting an order, the subscribed email automatically received an Order Deleted Notification with the order details.

7. Freestyle Enhancement – Amazon SQS Dead Letter Queue

Service Selected

Amazon SQS (Simple Queue Service), configured as a Dead Letter Queue (DLQ).

Why It Was Selected

The asynchronous delete flow relies on EventBridge delivering the OrderDeleted event to the notification and backup Lambdas. If a target fails to execute (for example, a transient error), the event could be lost. A Dead Letter Queue adds reliability by capturing any failed event so it is not lost and can be inspected or reprocessed.

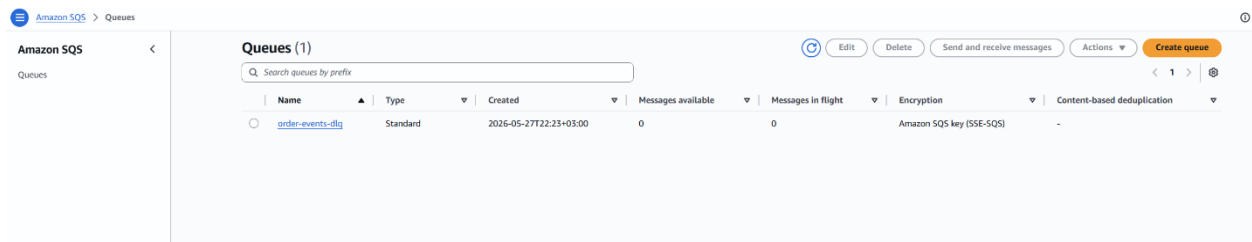
Functionality It Adds

Failed asynchronous events are automatically routed to the SQS Dead Letter Queue instead of disappearing. This protects against data loss in the event-driven pipeline and demonstrates a real-world resilience pattern.

How to Execute and Verify

- The DLQ is wired to the asynchronous targets, so no manual action is needed during normal use.
- To verify, open the SQS console (or run: `aws sqs list-queues`) and view the queue; failed messages appear there.
- The enhancement is described in the web client under the "Freestyle Enhancement" panel so it is visible from the UI.

Screenshot



8. Delete Order Code (Lambda Function)

The delete handler removes the order from DynamoDB and publishes an OrderDeleted event to EventBridge, then returns immediately so the deletion is asynchronous from the user's perspective.

```
import json, boto3, os

dynamodb = boto3.resource('dynamodb')
events = boto3.client('events')
table = dynamodb.Table(os.environ['ORDERS_TABLE'])
bus_name = os.environ['EVENT_BUS_NAME']

def lambda_handler(event, context):
    try:
        order_id = event['pathParameters']['orderId']
        resp = table.get_item(Key={'orderId': order_id})
        if 'Item' not in resp:
            return {'statusCode': 404,
                    'headers': {'Access-Control-Allow-Origin': '*'},
                    'body': 'Order not found'}
        order = resp['Item']
        table.delete_item(Key={'orderId': order_id})
        events.put_events(Entries=[{
            'Source': 'com.ordermanagement',
            'DetailType': 'OrderDeleted',
            'Detail': json.dumps(order, default=str),
            'EventBusName': bus_name
        }])
        return {'statusCode': 200,
                'headers': {'Access-Control-Allow-Origin': '*'},
                'body': json.dumps({'message': 'Order deleted and event published successfully'})}
    except Exception as e:
        return {'statusCode': 500,
                'headers': {'Access-Control-Allow-Origin': '*'},
                'body': str(e)}
```

Notification Lambda (triggered by EventBridge)

For completeness, the function that EventBridge invokes to send the email notification through SNS:

```
import json, boto3, os

sns = boto3.client('sns')

def lambda_handler(event, context):
    order = event['detail']
    msg = (f"Order {order['orderId']} was deleted. "
          f"Price: {order['price']}. "
          f"Desc: {order.get('orderDescription')}")
    sns.publish(TopicArn=os.environ['SNS_TOPIC_ARN'],
               Message=msg,
               Subject="Order Deleted Notification")
```